Projeto KUIDAMOS – CIExpert

Planejamento técnico dos módulos de hardware

Documento compartilhado entre os Grupos 6a, 6b e 6c.

1 Contexto do projeto

A KUIDAMOS é uma startup de visão computacional embarcada voltada à segurança urbana, parceira do programa CIExpert, de formação em microeletrônica. A empresa atua em aplicações como leitura automática de placas (LPR), detecção de pessoas, detecção de permanência ou perambulação e detecção e captura de faces.

A plataforma de referência da startup é a placa Luckfox Pico Pro Max, que combina processamento de vídeo e NPU. A aquisição de imagens pode ocorrer por rede (câmeras IP, protocolos como RTSP) ou diretamente de um sensor. O objetivo da parceria é avaliar uma solução de baixo custo para análise inteligente de imagens com processamento local em hardware embarcado, priorizando leitura de placas, detecção de permanência e captura de faces.

O objetivo acadêmico do programa é complementar esse esforço no nível de hardware: desenvolver, com as ferramentas Synopsys, módulos digitais bem delimitados que possam compor, no futuro, arquiteturas maiores de processamento de vídeo e visão computacional.

2 Organização do projeto

Três subgrupos trabalham de forma paralela, cada um responsável por um IP de hardware independente. Cada módulo é desenvolvido, verificado e avaliado por conta própria, sem dependência de integração entre os grupos.

Tabela 1: Organização dos subgrupos e respectivos módulos.

Grupo	Módulo	Domínio
6a	Bloco de multiplicação matricial 4×4 (int8 → int32)	Aritmética para redes neurais
6b	Acelerador de FFT	Processamento de áudio
6c	Front-end de pré-processamento de pixels	Processamento de imagem em streaming

Todos os grupos seguem o mesmo fluxo de projeto: RTL em SystemVerilog, verificação funcional, síntese lógica, DFT e implementação física com ferramentas Synopsys.

3 Diretrizes comuns

Natureza das entregas. Cada grupo entrega um IP educacional: funcional, sintetizável, verificado por testbench, documentado e avaliado com as ferramentas Synopsys pertinentes, com estrutura e interfaces claras o bastante para futura adaptação e reuso pela startup.

Prioridade de projeto. Módulos pequenos, verificáveis e concluídos dentro do prazo de aproximadamente cinco meses têm precedência sobre sofisticação arquitetural.

Fora do escopo comum. Inferência completa, substituição da NPU da plataforma, desenvolvimento de um acelerador completo, integração obrigatória entre os grupos e dependência da plataforma Luckfox ou de protocolos de vídeo.

Métricas de implementação. Área, frequência e potência serão medidas e reportadas a partir dos relatórios das ferramentas, sem metas numéricas pré-definidas.

---

4 Grupo 6a – Bloco de multiplicação matricial 4×4

4.1 Equipe

Integrante	**Trilha**
Felipe Lopes Gibin Duarte	DFT
Gabriel Santos Pinto	RTL
Giovana Ferreira Santos	Verification
Leonardo Santiago Melgaço Silva	Physical

4.2 Proposta e relevância

O Grupo 6a desenvolverá um bloco de hardware para multiplicação matricial de dimensão 4×4, com operandos inteiros de 8 bits com sinal e resultados de 32 bits com sinal. O módulo será especificado como um IP digital independente, implementado em SystemVerilog e submetido às etapas de verificação funcional, síntese lógica, projeto para testabilidade (DFT) e implementação física.

A multiplicação matricial constitui uma operação fundamental em redes neurais artificiais, sendo empregada em camadas lineares, operações de convolução após transformação para uma representação matricial e mecanismos de atenção. Em aceleradores de inferência, a utilização de operandos de 8 bits com sinal e acumulação em larguras superiores permite explorar paralelismo aritmético, mantendo uma representação compacta dos dados de entrada.

A implementação proposta tem como objetivo avaliar o comportamento funcional e as características de implementação de uma unidade de multiplicação matricial inteiramente paralela. A arquitetura será dimensionada para calcular simultaneamente os 16 elementos da matriz de saída, sem compartilhamento temporal dos recursos aritméticos entre elementos distintos da matriz.

O projeto será conduzido com ênfase em especificação verificável, reprodutibilidade dos resultados e compatibilidade com o fluxo de ferramentas EDA disponibilizado pelo programa CIExpert. Os resultados de síntese e implementação física permitirão caracterizar o custo de área e o comportamento temporal da solução adotada, sem estabelecer, nesta etapa, metas numéricas de área ou potência.

4.3 Especificação funcional

O bloco implementará o produto de duas matrizes quadradas A e B, ambas de dimensão 4×4, produzindo a matriz de saída C, também de dimensão 4×4. A operação será definida por

$$C[i, j] = \sum_{k=0}^{3} A[i, k] \cdot B[k, j], \qquad i, j \in \{0, 1, 2, 3\}. \tag{1}$$

Os elementos de A e B serão representados por inteiros de 8 bits em complemento de dois, pertencentes ao intervalo

$$-128 \leq A[i, j], B[i, j] \leq 127. \tag{2}$$

Cada elemento de C será representado por um inteiro de 32 bits com sinal. A implementação deverá preservar a interpretação signed dos operandos durante as multiplicações e a soma dos produtos parciais, garantindo a extensão de sinal e as larguras aritméticas necessárias para a obtenção do resultado especificado.

O maior módulo possível para um produto individual é

$$\max |A[i, k] \cdot B[k, j]| = 16384. \tag{3}$$

Considerando os quatro produtos que compõem cada elemento da matriz de saída, os limites exatos da soma são

$$-65024 \leq C[i, j] \leq 65536. \tag{4}$$

Esses limites estão contidos no intervalo representável por um inteiro de 32 bits com sinal. Portanto, para todas as combinações válidas de operandos de entrada, o resultado matemático poderá ser representado sem overflow de 32 bits, dispensando lógica de saturação.

A interface de controle utilizará os sinais `start` e `done`. O sinal `start` indicará a solicitação de uma operação, enquanto `done` indicará a conclusão do cálculo e a disponibilidade do resultado registrado. Os detalhes de temporização desses sinais deverão ser definidos de maneira consistente no RTL e verificados por meio de assertions.

4.4 Arquitetura inicial

A arquitetura inicial será definida pelas seguintes características:

- **Organização aritmética:** arquitetura totalmente paralela, com unidades de multiplicação e lógica de soma suficientes para calcular os 16 elementos da matriz de saída.

Cada elemento $C[i, j]$ será obtido pela soma de quatro produtos, sem compartilhamento temporal dos recursos aritméticos entre elementos distintos da matriz.

- **Carregamento dos operandos:** carregamento integral e simultâneo das duas matrizes de entrada. Cada matriz será representada por um barramento de 128 bits, totalizando 256 bits de operandos. Os dados de ambas as matrizes serão capturados e registrados na mesma borda ativa do clock, garantindo que o cálculo utilize um conjunto coerente de operandos registrados.
- **Armazenamento interno:** utilização de registradores para armazenar os operandos capturados e os 16 resultados de 32 bits. A matriz de saída possuirá, portanto, 512 bits de largura total. A implementação não dependerá de RAM externa ou de macros de memória.
- **Interface de controle:** interface síncrona customizada, baseada nos sinais `clk`, `start` e `done`, além dos barramentos de entrada e saída. O sinal `start` comandará a captura dos operandos e o início da operação. O sinal `done` indicará a conclusão do processamento, conforme o protocolo de controle estabelecido na especificação de RTL.
- **Microarquitetura sem pipeline:** não serão utilizados estágios de pipeline nem escalonamento temporal de operações aritméticas. O cálculo dos 16 elementos da matriz de saída ocorrerá em uma única etapa combinacional, a partir dos operandos registrados.
- **Temporização e latência:** adotando-se a borda ativa do clock em que os operandos são capturados como referência temporal, os resultados serão registrados na borda ativa seguinte. A latência entre a captura dos operandos e o registro dos resultados será, portanto, de um ciclo de clock. Essa definição não inclui eventuais atrasos externos à interface, nem substitui a verificação dos requisitos de setup e hold da implementação.
- **Escopo arquitetural inicial:** a primeira versão não contemplará tiling, arranjos sistólicos, processamento em streaming ou mecanismos de reutilização temporal de multiplicadores. Essas extensões poderão ser avaliadas futuramente, desde que não alterem os requisitos funcionais estabelecidos para a versão inicial.

A microarquitetura será implementada de forma a manter explícita a separação entre os registradores de entrada, a lógica combinacional de multiplicação e acumulação e os registradores de saída. A descrição RTL deverá permitir que as ferramentas de síntese identifiquem a estrutura combinacional sem inferência intencional de elementos de memória adicionais.

A interface e o comportamento temporal deverão ser formalizados em conjunto com o RTL. Em particular, a especificação deverá determinar as condições de captura associadas a `start`, o instante de ativação de `done`, a validade dos resultados e o comportamento dos sinais após reset. Esses aspectos serão tratados como requisitos verificáveis, e não como decisões implícitas da implementação.

4.5 Escopo

Dentro do escopo:

- Especificação funcional e temporal do produto matricial 4×4, incluindo representação numérica, larguras dos operandos e resultados e regras de extensão de sinal.
- Definição da interface síncrona, com carregamento paralelo das duas matrizes de entrada e sinais de controle `start` e `done`.
- Desenvolvimento de RTL sintetizável em SystemVerilog, com arquitetura combinacional paralela e sem pipeline, além de registradores de entrada e saída.
- Desenvolvimento de modelo de referência independente em Python/NumPy para cálculo dos resultados esperados e geração de vetores de teste em arquivos de texto (`.txt`).
- Desenvolvimento de testbench em SystemVerilog, com leitura dos vetores de teste, aplicação de estímulos, Scoreboard para comparação automatizada dos resultados e SystemVerilog Assertions (SVA) para verificação dos requisitos temporais e funcionais da interface.
- Verificação funcional de casos nominais, valores positivos e negativos, operandos nulos, valores extremos e combinações direcionadas e aleatórias.
- Síntese lógica com restrição inicial de clock de 100 MHz, correspondente a um período de 10 ns, e avaliação dos relatórios de área e temporização.
- Execução do fluxo de DFT, incluindo verificações estruturais, análise de regras de projeto para testabilidade e inserção de scan, conforme os recursos disponíveis no ambiente.
- Implementação física, análise dos relatórios gerados pelas ferramentas e documentação das decisões arquiteturais, restrições de projeto e resultados obtidos.

Fora do escopo:

- Implementação de uma cadeia completa de inferência de redes neurais ou substituição da NPU existente na plataforma da startup.
- Desenvolvimento de um acelerador completo de visão computacional ou integração obrigatória com os módulos dos Grupos 6b e 6c.
- Dependência funcional da plataforma Luckfox Pico Pro Max, de sensores de imagem, de protocolos de vídeo ou de interfaces externas de comunicação.
- Arquiteturas com pipeline, compartilhamento temporal de recursos aritméticos, execução iterativa ou processamento em streaming na versão inicial.
- Definição antecipada de metas rígidas de área e potência, que serão avaliadas a partir dos resultados obtidos no fluxo de implementação.

4.6 Metodologia e ferramentas

O desenvolvimento seguirá um fluxo incremental de engenharia, no qual cada etapa produzirá artefatos verificáveis e resultados documentados. A metodologia deverá assegurar consistência entre a especificação funcional, o modelo de referência, o RTL e os resultados obtidos nas ferramentas EDA.

1. **Especificação funcional e temporal.**

   Formalização da equação do produto matricial, da representação signed em complemento de dois, das larguras aritméticas e dos limites numéricos. Também serão definidos o protocolo de `start`/`done`, as condições de captura simultânea dos operandos, o comportamento do reset e a latência de um ciclo entre a captura e o registro dos resultados.

2. **Modelo de referência e geração de vetores.**

   Desenvolvimento de um Golden Model em Python utilizando NumPy, independente da implementação RTL. O modelo calculará os produtos matriciais esperados e gerará vetores de entrada e saída em arquivos `.txt`, em formato documentado e compatível com a leitura pelo testbench. A geração incluirá casos direcionados para operandos nulos, valores positivos e negativos, extremos numéricos e combinações aleatórias reproduzíveis.

3. **Implementação RTL.**

   Descrição do bloco em SystemVerilog, contemplando registradores para as matrizes de entrada e para os resultados, lógica combinacional paralela de multiplicação e acumulação e controle síncrono por meio dos sinais `start` e `done`. A implementação deverá respeitar a latência especificada, utilizar larguras e extensões de sinal explícitas e evitar construções incompatíveis com síntese.

4. **Verificação funcional.**

   Desenvolvimento de testbench em SystemVerilog com leitura dos arquivos de vetores gerados pelo Golden Model. Um Scoreboard comparará automaticamente cada elemento da matriz de saída com o resultado esperado, registrando divergências e emitindo um resumo da execução. A verificação deverá contemplar, no mínimo, a correção numérica, a captura simultânea dos operandos, a latência de um ciclo, a indicação de conclusão e o comportamento dos registradores.

   Serão implementadas SystemVerilog Assertions (SVA) para verificar propriedades temporais da interface e dos registradores. As propriedades deverão ser consistentes com o protocolo efetivamente especificado, incluindo as condições de reset, os eventos de captura, a atualização dos resultados e a sinalização de conclusão. A execução deverá demonstrar a ausência de violações nas propriedades aplicáveis aos cenários de teste.

5. **Síntese lógica.**

   Execução da síntese com o Synopsys Design Compiler. Como ponto de partida, será aplicada uma restrição de clock de 100 MHz, equivalente a um período de 10 ns, por meio de constraints temporais apropriadas. A análise incluirá área estimada, caminhos críticos, temporização e eventuais violações reportadas. O fechamento temporal não será presumido: deverá ser demonstrado por meio dos relatórios produzidos pela ferramenta.

6. **DFT e inserção de scan.**

   Utilização do Synopsys TestMAX DFT para análise de testabilidade, execução das verificações estruturais pertinentes, incluindo `dft_drc`, pré-visualização da inserção de scan com `preview_dft` e execução das etapas de inserção de scan suportadas pelo ambiente. Os resultados e eventuais limitações deverão ser registrados.

7. **Implementação física.**

Execução do fluxo de implementação com o Synopsys Fusion Compiler (`fc_shell`), conforme a configuração disponível no CIExpert. Serão documentados os resultados relevantes de implementação, área, temporização e potência, bem como as restrições ou limitações encontradas durante o processamento.

8. **Análise e documentação.**

   Consolidação dos resultados de verificação, síntese, DFT e implementação física. A documentação final deverá correlacionar as decisões arquiteturais com os resultados obtidos, registrar as configurações utilizadas e permitir a reprodução do fluxo executado.

Restrições de projeto e ambiente tecnológico.

A restrição inicial de temporização será definida por um clock de 100 MHz, com período de 10 ns. Essa restrição será utilizada como referência para a análise dos caminhos temporais durante a síntese e as etapas subsequentes de implementação. As constraints deverão identificar o clock principal e contemplar os demais parâmetros temporais exigidos pela metodologia do ambiente, conforme aplicável.

O processo tecnológico (PDK) e a biblioteca de células serão os disponibilizados pelo ambiente educacional CIExpert. A identificação exata do nó tecnológico, da biblioteca e de suas versões permanece pendente de confirmação no ambiente. Esses parâmetros deverão ser registrados antes da execução definitiva do fluxo de síntese e implementação física, para assegurar consistência e reprodutibilidade dos resultados.

Ferramentas e artefatos principais.

As ferramentas e linguagens previstas são SystemVerilog, Python/NumPy, Synopsys Design Compiler, Synopsys TestMAX DFT e Synopsys Fusion Compiler. Os artefatos principais incluem código RTL, Golden Model, arquivos de vetores `.txt`, testbench, Scoreboard, propriedades SVA, arquivos de constraints, scripts de execução e relatórios das ferramentas.

A utilização de versões específicas das ferramentas, bibliotecas de células, scripts de referência e parâmetros do fluxo seguirá a configuração disponibilizada pelo programa CIExpert. As versões efetivamente utilizadas deverão ser registradas na documentação final.

4.7 Entregáveis

1. **Especificação técnica:** descrição funcional e temporal do módulo, incluindo formato dos dados, interface, protocolo de controle, latência e condições de reset.

2. **RTL sintetizável:** implementação em SystemVerilog do bloco de multiplicação matricial 4×4, com arquitetura paralela sem pipeline e registradores de entrada e saída.

3. **Golden Model e vetores de teste:** código Python/NumPy e arquivos `.txt` contendo operandos e resultados esperados, acompanhados da descrição do formato utilizado.

4. **Ambiente de verificação:** testbench em SystemVerilog, Scoreboard, propriedades SVA e relatório de execução com registro dos casos testados, divergências encontradas e resultado final da verificação.

5. **Resultados de síntese:** scripts, constraints aplicadas e relatórios relevantes do Synopsys Design Compiler, incluindo área e análise de temporização.
6. **Resultados de DFT:** relatórios das verificações estruturais, análise de testabilidade e inserção de scan, conforme suportado pelo fluxo disponibilizado.
7. **Resultados da implementação física:** arquivos e relatórios relevantes do Synopsys Fusion Compiler, incluindo métricas de área, temporização e potência quando disponíveis.
8. **Documentação final:** descrição da arquitetura, decisões de implementação, parâmetros tecnológicos, resultados experimentais, limitações identificadas e instruções necessárias para reproduzir o fluxo.

4.8 Critérios de aceitação

A conclusão do módulo será avaliada segundo os critérios funcionais, temporais, estruturais e metodológicos a seguir:

- **Correção aritmética:** para todas as combinações válidas de operandos, cada elemento da matriz de saída deverá corresponder à soma dos quatro produtos definidos na especificação, com interpretação signed correta e resultado representado em 32 bits.
- **Captura paralela dos operandos:** as duas matrizes de 128 bits deverão ser capturadas e registradas simultaneamente na borda de clock definida pelo protocolo de controle. O cálculo deverá utilizar os operandos registrados correspondentes à mesma operação.
- **Latência e ausência de pipeline:** o resultado calculado deverá ser registrado na borda ativa de clock imediatamente seguinte à captura dos operandos, caracterizando latência de cálculo de um ciclo. A arquitetura deverá ser compatível com o modelo especificado de uma única etapa combinacional, sem pipeline e sem execução iterativa dos produtos.
- **Protocolo de interface:** os sinais `start` e `done` deverão obedecer ao protocolo temporal documentado. A conclusão deverá ser sinalizada de forma consistente com a disponibilidade dos resultados registrados, e o comportamento após reset deverá corresponder à especificação.
- **Verificação automatizada:** o testbench deverá ler os vetores de teste gerados pelo Golden Model, comparar automaticamente os 16 elementos da saída por meio do Scoreboard e reportar eventuais divergências. A execução deverá incluir casos-limite e demonstrar concordância entre o RTL e o modelo de referência para todos os vetores executados.
- **Propriedades temporais:** as assertions SVA aplicáveis deverão verificar os requisitos de controle, captura dos operandos, latência, atualização dos resultados e comportamento dos registradores. Não deverão permanecer violações não justificadas nas propriedades consideradas obrigatórias para a entrega.
- **Sintetizabilidade:** o RTL deverá ser aceito pelo fluxo de síntese, sem erros impeditivos ou inferências estruturais incompatíveis com a arquitetura especificada. Eventuais avisos relevantes deverão ser analisados e documentados.

- **Análise temporal:** a síntese deverá ser executada com a restrição inicial de clock de 100 MHz. Os relatórios deverão apresentar os resultados da análise temporal, incluindo eventuais violações da restrição. A aceitação da execução da análise não implica, por si só, garantia de fechamento temporal.
- **DFT e implementação física:** as etapas previstas de DFT, inserção de scan e implementação física deverão ser executadas de forma reproduzível, na extensão suportada pelo ambiente. Resultados incompletos ou limitações do fluxo deverão ser explicitamente documentados, sem serem apresentados como etapas concluídas.
- **Rastreabilidade e documentação:** os parâmetros de clock, PDK, biblioteca de células, versões de ferramentas, scripts, vetores de teste e relatórios deverão ser identificados de modo a permitir a reprodução das análises.
- **Relatório de métricas:** os resultados de área, temporização e potência deverão ser reportados com base nos dados efetivamente produzidos pelas ferramentas. Não será considerada demonstrada qualquer meta de desempenho que não esteja sustentada pelos relatórios correspondentes.

4.9 Riscos e mitigação

Tabela 2: Riscos e estratégias de mitigação.

Risco	**Mitigação**
Ambiguidade na especificação numérica	Formalização das larguras, da interpretação signed, dos limites de representação e da equação de referência antes da implementação RTL.
Inconsistência entre o Golden Model e o RTL	Implementação independente do modelo Python/NumPy, geração de vetores reproduzíveis e comparação automatizada por Scoreboard.
Erros no protocolo de controle	Definição explícita dos eventos de captura e conclusão, acompanhada de assertions SVA para os requisitos temporais da interface.
Caminho combinacional crítico excessivamente longo	Análise dos caminhos críticos com clock de referência de 100 MHz e avaliação dos relatórios de síntese. Eventuais ajustes deverão preservar os requisitos funcionais e a ausência de pipeline.
Dificuldades no uso das ferramentas EDA	Validação antecipada do ambiente CIExpert, execução de um fluxo mínimo ponta a ponta e registro das versões, bibliotecas e configurações utilizadas.
Indisponibilidade ou indefinição do PDK	Confirmação do PDK e da biblioteca de células educacionais antes da síntese definitiva e da implementação física.

Continua na próxima página

Tabela 2 – continuação

Risco	Mitigação
Complexidade na inserção de scan	Execução incremental das verificações DFT, análise das regras de testabilidade e documentação de limitações ou incompatibilidades identificadas.
Cobertura insuficiente de casos-limite	Inclusão de testes direcionados para valores extremos, operandos nulos, sinais mistos e vetores aleatórios, com comparação contra o modelo de referência.

5 Grupo 6b – Acelerador de FFT para áudio

5.1 Equipe

Integrante	Trilha
Gabriel da Conceição Miranda	DFT
Roberto Rodrigues Macedo Junior	RTL
Ryan Guilherme Morais Nascimento	RTL

5.2 Quais são os blocos importantes para implementar o projeto?

O bloco selecionado é um **Acelerador de Transformada Rápida de Fourier (FFT)** focado na extração de características espectrais (*feature extraction*) de sinais de áudio. O objetivo é converter as amostras de áudio recebidas da rede via RTSP do domínio do tempo para o domínio da frequência. Esse espectro é a assinatura acústica crucial para que a NPU classifique eventos de segurança pública (como estampidos de arma de fogo ou quebra de vidro).

Viabilidade em 5 meses: A complexidade é mantida sob controle ao optarmos pelo desenvolvimento de um acelerador *hardwired* de tamanho fixo (ex: janela de 64 ou 256 pontos), ao invés de projetar um Processador Digital de Sinais (DSP) de propósito geral e instrução programável.

5.3 Qual é a interface do bloco (entradas/saídas)?

A interface é projetada para processamento contínuo (*streaming*):

- **Entradas:** Sinais de sincronismo global (Clock e Reset), Barramento de dados de áudio (Amostras digitais PCM oriundas do *payload* RTP), e sinal de controle de fluxo de entrada (`enable` / `valid_in`).
- **Saídas:** Barramento de dados espectrais (magnitudes das frequências processadas), e sinalização de status (`data_ready` / `valid_out`) para notificar a NPU ou a memória de que a conversão da janela foi concluída.

5.4 O que vai dentro e o que vai fora do chip?

- **Dentro (Escopo do IP):** O caminho de dados (*datapath*) contendo as unidades borboleta (*butterfly units*) com os somadores e multiplicadores do algoritmo, a máquina de estados (FSM) de controle sequencial, e registradores temporários de *pipeline* para segurar as amostras em trânsito.
- **Fora (A cargo do Sistema/Placa):** O tráfego de rede e desempacotamento de protocolo (RTSP/RTP), que será gerenciado via software (Linux/CPU); os microfones físicos de captação; e a memória externa ou tabelas (ROM) contendo os coeficientes trigonométricos (*twiddle factors*) da FFT.

6 Grupo 6c – Front-End de Pixels (Streaming e Redimensionamento)

6.1 Equipe

Integrante	**Trilha**
Wagner Cutrim Rabelo Junior	DFT
Thiago Fernandes Carvalho de Souza	RTL

6.2 Proposta e relevância

O Grupo 6c desenvolverá um **Front-End de Pré-Processamento de Pixels** para imagens em fluxo contínuo (*streaming*). O módulo será especificado como um IP digital independente, implementado em SystemVerilog e submetido às etapas de verificação funcional, síntese lógica, projeto para testabilidade (DFT) e implementação física.

O bloco receberá pixels RGB previamente decodificados e executará uma sequência de operações simples de pré-processamento: conversão para escala de cinza (*grayscale*), seleção de uma região de interesse (*crop*), redimensionamento por vizinho mais próximo (*nearest-neighbor resize*) e limiarização (*threshold*). A saída será um fluxo de pixels de 8 bits acompanhado dos sinais necessários para identificar a validade dos dados e a estrutura de linhas e quadros.

A utilização de um front-end dedicado permite deslocar para hardware operações repetitivas que normalmente seriam executadas antes de etapas mais complexas de processamento de imagem ou inferência. O objetivo acadêmico não é substituir a NPU existente na plataforma de referência, mas avaliar a implementação de um módulo de pré-processamento compacto, sintetizável e verificável, compatível com o fluxo de ferramentas EDA disponibilizado pelo programa CIExpert.

A escolha dessas operações procura manter o escopo compatível com o prazo de aproximadamente cinco meses. A arquitetura será baseada em aritmética inteira, comparações, contadores e lógica de controle, evitando algoritmos de interpolação de maior custo, armazenamento de quadros completos e dependência de protocolos de vídeo ou de uma câmera específica.

Os resultados de síntese, DFT e implementação física permitirão caracterizar o custo de área, o comportamento temporal e as características estruturais da solução adotada, sem estabelecer nesta etapa metas rígidas de área, potência ou desempenho.

6.3 Especificação funcional

A entrada de vídeo será representada inicialmente no formato RGB888. Cada pixel será composto por três componentes de 8 bits sem sinal:

$$R, G, B \in \{0, 1, \ldots, 255\}. \tag{5}$$

Cada pixel de entrada possuirá, portanto, 24 bits de informação de cor. O processamento será realizado somente quando o sinal de validade de entrada indicar a presença de um pixel válido.

A primeira operação será a conversão de RGB para escala de cinza. Como referência inicial, será utilizada uma aproximação inteira da luminância:

$$Y = \left\lfloor \frac{77R + 150G + 29B}{256} \right\rfloor, \tag{6}$$

onde Y será representado por um valor sem sinal de 8 bits. Os coeficientes inteiros foram escolhidos de forma que a soma seja igual a 256, permitindo implementar a divisão final por meio de deslocamento binário e evitando aritmética de ponto flutuante.

O estágio de *crop* selecionará uma região retangular de interesse dentro da imagem. A região será definida por uma posição inicial (x_{start}, y_{start}) e pelas dimensões W_{crop} e H_{crop}. Um pixel de coordenadas (x, y) pertencerá à região válida quando satisfizer simultaneamente:

$$x_{start} \leq x < x_{start} + W_{crop} \tag{7}$$

$$y_{start} \leq y < y_{start} + H_{crop}. \tag{8}$$

Pixels localizados fora da região configurada serão descartados pelo fluxo de saída. Os limites válidos dos parâmetros deverão ser definidos de acordo com as dimensões da imagem de entrada e verificados no ambiente de teste.

O estágio de redimensionamento utilizará o algoritmo de vizinho mais próximo. Para uma imagem de entrada de dimensões $W_{in} \times H_{in}$ e uma imagem de saída de dimensões $W_{out} \times H_{out}$, o mapeamento conceitual de coordenadas será dado por:

$$x_{src} = \left\lfloor \frac{x_{out} W_{in}}{W_{out}} \right\rfloor \tag{9}$$

$$y_{src} = \left\lfloor \frac{y_{out} H_{in}}{H_{out}} \right\rfloor. \tag{10}$$

A primeira versão priorizará operações de redução de resolução (*downscaling*). Essa restrição permite implementar o redimensionamento por seleção e descarte de pixels e

linhas, evitando a necessidade de reconstrução por interpolação e reduzindo os requisitos de armazenamento interno.

A etapa de limiarização será configurável. Para um limiar T de 8 bits, a saída binária será definida por:

$$P_{out} = \begin{cases} 255, & Y \geq T, \\ 0, & Y < T. \end{cases} \tag{11}$$

O estágio de *threshold* poderá ser habilitado ou desabilitado. Quando desabilitado, o valor em escala de cinza resultante dos estágios anteriores será mantido na saída. Essa possibilidade permite utilizar o mesmo IP em cenários que exigem uma imagem em tons de cinza ou uma imagem binária.

A ordem funcional inicial do processamento será:

$$\text{RGB} \rightarrow \text{Grayscale} \rightarrow \text{Crop} \rightarrow \text{Resize Nearest} \rightarrow \text{Threshold} \rightarrow \text{Saída}.$$

O comportamento dos marcadores de linha e quadro após operações de *crop* e *resize* será especificado de forma explícita antes da implementação definitiva do RTL. Os sinais de saída deverão corresponder às dimensões da imagem efetivamente produzida pelo IP, e não às dimensões originais de entrada.

6.4 Arquitetura inicial

A arquitetura inicial será definida pelas seguintes características:

- **Processamento em streaming:** os pixels serão recebidos sequencialmente, acompanhados por sinais de validade e sincronismo. O processamento ocorrerá à medida que os dados percorrem o bloco, sem armazenamento de um quadro completo em memória interna.
- **Contadores espaciais:** contadores de coordenadas X e Y identificarão a posição do pixel corrente dentro da imagem de entrada. Esses contadores serão atualizados somente para pixels considerados válidos pela interface.
- **Conversão RGB para grayscale:** o datapath utilizará multiplicações por constantes, somas e deslocamento para produzir um valor de 8 bits. A implementação deverá preservar largura intermediária suficiente para impedir overflow antes da redução para a saída de 8 bits.
- **Crop configurável:** comparadores de coordenadas determinarão se o pixel corrente pertence à região de interesse configurada. Pixels externos à janela serão descartados sem necessidade de armazenamento adicional.
- **Resize nearest-neighbor:** o redimensionamento será implementado por lógica de seleção baseada nas dimensões de entrada e saída. A primeira versão será orientada a redução de resolução, evitando interpolação bilinear, bicúbica ou armazenamento completo de linhas e quadros.
- **Threshold configurável:** um comparador de 8 bits implementará a limiarização. Um sinal de habilitação permitirá utilizar ou ignorar esse estágio.

- **Interface de configuração:** parâmetros como dimensões da imagem, coordenadas de crop, dimensões de saída e valor de threshold serão fornecidos por sinais ou registradores de configuração definidos no RTL. A forma final de carregamento desses parâmetros será documentada antes da verificação definitiva.
- **Armazenamento interno limitado:** a arquitetura evitará framebuffer completo e, na primeira versão, não dependerá de RAM externa ou de macros de memória. Registradores serão utilizados apenas para dados de pipeline, parâmetros de configuração, contadores e estados de controle estritamente necessários.
- **Modularidade:** grayscale, crop, resize e threshold serão descritos de maneira separável no RTL, permitindo testes unitários e integração incremental do pipeline.

A interface inicial será síncrona e orientada a fluxo de pixels. Os principais sinais previstos são:

- `clk`: clock principal;
- `reset_n`: reset do módulo;
- `pixel_r[7:0]`, `pixel_g[7:0]` e `pixel_b[7:0]`: componentes RGB do pixel de entrada;
- `pixel_valid`: indicação de pixel de entrada válido;
- sinais de início/fim de linha e início/fim de quadro;
- parâmetros de configuração para dimensões de entrada, região de crop, dimensões de saída e threshold;
- `pixel_out[7:0]`: pixel processado de saída;
- `pixel_out_valid`: indicação de pixel de saída válido;
- sinais correspondentes de início/fim de linha e início/fim de quadro na saída.

Os nomes definitivos dos sinais e o comportamento ciclo a ciclo serão formalizados juntamente com o RTL. Em particular, deverão ser especificados o comportamento após reset, a atualização dos contadores, a relação entre validade de entrada e saída e a regeneração dos marcadores de linha e quadro após crop e resize.

O IP não realizará aquisição de vídeo, decodificação H.264/H.265, comunicação RTSP/RTP, Ethernet ou interface direta com sensor de imagem. O ponto de entrada funcional do bloco será um fluxo de pixels já decodificados.

6.5 Escopo

Dentro do escopo:

- Especificação funcional e temporal do pipeline de pré-processamento, incluindo formato dos pixels, sinais de sincronismo, parâmetros configuráveis e comportamento das operações.
- Definição da interface síncrona de entrada e saída para fluxo de pixels RGB888 e pixels processados de 8 bits.

- Desenvolvimento de RTL sintetizável em SystemVerilog para os submódulos de grayscale, crop, resize nearest-neighbor e threshold.
- Implementação de contadores de coordenadas e lógica de controle necessária para acompanhar linhas, quadros e regiões de interesse.
- Desenvolvimento de modelo de referência independente em Python utilizando NumPy e, quando conveniente, OpenCV para processamento das imagens e geração dos resultados esperados.
- Conversão de imagens de teste em vetores de pixels compatíveis com a leitura pelo testbench e reconstrução da saída para comparação visual e automática.
- Desenvolvimento de testbench em SystemVerilog com Scoreboard para comparação automatizada entre o resultado do RTL e o Golden Model.
- Utilização de SystemVerilog Assertions (SVA) para verificar requisitos temporais e funcionais da interface, incluindo reset, validade dos dados e sincronismo de linhas e quadros.
- Verificação funcional de casos nominais, imagens pequenas direcionadas, valores extremos de RGB, diferentes regiões de crop, diferentes fatores de redução e valores de threshold.
- Síntese lógica com restrição inicial de clock de 100 MHz, correspondente a um período de 10 ns, e avaliação dos relatórios de área e temporização.
- Execução do fluxo de DFT, incluindo verificações estruturais, análise de regras de projeto para testabilidade e inserção de scan, conforme os recursos disponíveis no ambiente.
- Implementação física, análise dos relatórios gerados pelas ferramentas e documentação das decisões arquiteturais, restrições de projeto e resultados obtidos.

Fora do escopo:

- Implementação ou treinamento de modelos completos de inteligência artificial.
- Substituição da NPU presente na plataforma de referência da startup.
- Implementação de uma cadeia completa de inferência ou de um acelerador completo de visão computacional.
- Integração obrigatória com os módulos desenvolvidos pelos Grupos 6a e 6b.
- Dependência funcional da plataforma Luckfox Pico Pro Max, de uma câmera específica ou de um sensor de imagem específico.
- Implementação de protocolos RTSP, RTP, Ethernet ou MIPI CSI-2.
- Decodificação de vídeo H.264/H.265 e processamento de bitstream comprimido.
- Funções de ISP associadas ao sensor, como debayering, correção de exposição, balanço de branco ou processamento de imagem RAW.

- Armazenamento de quadros completos em memória interna ou desenvolvimento de controladores de DRAM.
- Interpolação bilinear, bicúbica ou outros métodos avançados de redimensionamento na versão inicial.
- Metas rígidas antecipadas de área e potência, que serão avaliadas a partir dos resultados obtidos nas ferramentas.

6.6 Metodologia e ferramentas

O desenvolvimento seguirá um fluxo incremental de engenharia, no qual cada estágio funcional será especificado e verificado antes da consolidação do pipeline completo. A metodologia deverá manter consistência entre a especificação, o Golden Model, o RTL e os resultados obtidos nas ferramentas EDA.

1. **Especificação funcional e temporal.**

   Definição do formato RGB888 de entrada, formato de saída de 8 bits, dimensões suportadas, parâmetros de crop, resize e threshold, sinais de sincronismo e comportamento do reset. Também será documentada a ordem das operações e o protocolo temporal da interface.

2. **Modelo de referência e geração de vetores.**

   Desenvolvimento de um Golden Model em Python utilizando NumPy e, quando aplicável, OpenCV. O modelo realizará independentemente as operações de grayscale, crop, resize nearest-neighbor e threshold. Imagens de teste serão convertidas em sequências de pixels e os resultados esperados serão armazenados em formato compatível com o testbench.

   A geração incluirá casos direcionados de dimensões reduzidas, padrões sintéticos, cores extremas, regiões de crop nas bordas e no centro da imagem, diferentes razões de redução e diferentes valores de threshold, além de imagens reais utilizadas apenas como estímulos de validação.

3. **Implementação RTL.**

   Desenvolvimento incremental dos submódulos em SystemVerilog. Inicialmente será validado um caminho de passagem direta (*pass-through*); em seguida serão incorporados grayscale, crop, resize e threshold. A implementação deverá evitar construções incompatíveis com síntese e utilizar larguras explícitas para as operações aritméticas intermediárias.

4. **Verificação funcional.**

   Desenvolvimento de testbench em SystemVerilog com leitura dos vetores gerados pelo Golden Model. Um Scoreboard comparará automaticamente os pixels válidos produzidos pelo RTL com os valores esperados e registrará divergências.

   Serão implementadas SystemVerilog Assertions para propriedades relacionadas ao reset, aos sinais de validade, aos marcadores de linha e quadro e à estabilidade dos parâmetros de configuração durante o processamento. A execução deverá demonstrar a ausência de violações não justificadas nos cenários considerados obrigatórios.

5. **Síntese lógica.**

   Execução da síntese com o Synopsys Design Compiler. Como ponto de partida, será aplicada uma restrição de clock de 100 MHz, equivalente a um período de 10 ns, por meio de constraints temporais apropriadas. A análise incluirá área estimada, caminhos críticos, temporização e eventuais violações reportadas.

   O fechamento temporal não será presumido; os resultados deverão ser demonstrados pelos relatórios produzidos pela ferramenta. Caso sejam necessários ajustes microarquiteturais, eles deverão preservar o comportamento funcional especificado.

6. **DFT e inserção de scan.**

   Utilização do Synopsys TestMAX DFT para análise de testabilidade, execução das verificações estruturais pertinentes, incluindo `dft_drc`, pré-visualização da inserção de scan com `preview_dft` e execução das etapas de inserção de scan suportadas pelo ambiente.

   A estratégia inicial priorizará uma arquitetura baseada em registradores e lógica combinacional, evitando macros de memória que introduzam requisitos adicionais de teste de memória. Os resultados, violações e limitações encontradas deverão ser documentados.

7. **Implementação física.**

   Execução do fluxo de implementação com o Synopsys Fusion Compiler (`fc_shell`), conforme a configuração disponível no CIExpert. Serão documentados os resultados relevantes de implementação, incluindo área, temporização e potência quando disponíveis.

8. **Análise e documentação.**

   Consolidação dos resultados de verificação funcional, síntese, DFT e implementação física. A documentação final deverá relacionar as decisões arquiteturais aos resultados obtidos, registrar as configurações utilizadas e permitir a reprodução do fluxo executado.

Restrições de projeto e ambiente tecnológico.

A restrição inicial de temporização será definida por um clock de 100 MHz, com período de 10 ns. Essa restrição será utilizada como referência inicial para a análise dos caminhos temporais durante a síntese e as etapas subsequentes de implementação. O atendimento à restrição deverá ser confirmado pelos relatórios das ferramentas.

O processo tecnológico (PDK) e a biblioteca de células serão os disponibilizados pelo ambiente educacional CIExpert. A identificação exata do nó tecnológico, da biblioteca e de suas versões permanece pendente de confirmação no ambiente. Esses parâmetros deverão ser registrados antes da execução definitiva do fluxo de síntese e implementação física.

A primeira versão assumirá processamento de imagens por fluxo de pixels previamente decodificados. A resolução máxima, as dimensões de crop permitidas e as relações de redução suportadas serão formalizadas durante a etapa de especificação, de forma a evitar a inclusão de funcionalidades não necessárias para a avaliação do IP.

Ferramentas e artefatos principais.

As ferramentas e linguagens previstas são SystemVerilog, Python, NumPy, OpenCV, Synopsys VCS, Synopsys Design Compiler, Synopsys TestMAX DFT e Synopsys Fusion

Compiler. O Synopsys Verdi poderá ser utilizado para inspeção e depuração de formas de onda conforme a disponibilidade do ambiente.

Os artefatos principais incluem código RTL, Golden Model, imagens de teste, arquivos de vetores, testbench, Scoreboard, propriedades SVA, arquivos de constraints, scripts de execução e relatórios das ferramentas.

A utilização de versões específicas das ferramentas, bibliotecas de células, scripts de referência e parâmetros do fluxo seguirá a configuração disponibilizada pelo programa CIExpert. As versões efetivamente utilizadas deverão ser registradas na documentação final.

6.7 Entregáveis

1. **Especificação técnica:** descrição funcional e temporal do módulo, incluindo formato dos pixels, interface, protocolo de fluxo, parâmetros configuráveis, dimensões suportadas, comportamento de reset e sincronismo de linhas e quadros.
2. **RTL sintetizável:** implementação em SystemVerilog do pipeline de pré-processamento e de seus submódulos de grayscale, crop, resize nearest-neighbor e threshold.
3. **Golden Model e vetores de teste:** código Python/NumPy/OpenCV, imagens de entrada, vetores de pixels e resultados esperados em formato documentado.
4. **Ambiente de verificação:** testbench em SystemVerilog, Scoreboard, propriedades SVA e relatório de execução com registro dos cenários testados e divergências encontradas.
5. **Resultados de síntese:** scripts, constraints aplicadas e relatórios relevantes do Synopsys Design Compiler, incluindo área e análise de temporização.
6. **Resultados de DFT:** relatórios das verificações estruturais, análise de testabilidade, inserção de scan e demais resultados suportados pelo ambiente.
7. **Resultados da implementação física:** arquivos e relatórios relevantes do Synopsys Fusion Compiler, incluindo métricas de área, temporização e potência quando disponíveis.
8. **Documentação final:** descrição da arquitetura, decisões de implementação, parâmetros tecnológicos, resultados experimentais, limitações identificadas e instruções necessárias para reproduzir o fluxo.

6.8 Critérios de aceitação

A conclusão do módulo será avaliada segundo os critérios funcionais, temporais, estruturais e metodológicos a seguir:

- **Correção da conversão grayscale:** para todos os vetores de teste considerados obrigatórios, o valor de saída do estágio deverá corresponder ao resultado produzido pelo Golden Model segundo a aproximação inteira especificada.
- **Correção do crop:** somente pixels cujas coordenadas pertençam à região configurada deverão ser considerados válidos na saída desse estágio. Casos de borda deverão ser exercitados explicitamente.

- **Correção do resize:** os pixels produzidos após o redimensionamento por vizinho mais próximo deverão corresponder ao mapeamento de coordenadas definido no Golden Model para as dimensões utilizadas nos testes.
- **Correção do threshold:** quando habilitado, o estágio deverá produzir 0 ou 255 de acordo com o limiar configurado; quando desabilitado, deverá preservar o valor de grayscale proveniente dos estágios anteriores.
- **Sincronismo de imagem:** os marcadores de linha e quadro deverão ser consistentes com a imagem efetivamente produzida após as operações de crop e resize. Não deverão ser gerados pixels válidos fora dos limites configurados.
- **Verificação automatizada:** o testbench deverá comparar automaticamente a sequência de pixels válidos da saída com o Golden Model e reportar eventuais divergências. A execução deverá incluir casos direcionados e demonstrar concordância para todos os vetores considerados obrigatórios.
- **Propriedades temporais:** as assertions SVA aplicáveis deverão verificar reset, validade dos dados, atualização dos contadores e sincronismo da interface. Não deverão permanecer violações não justificadas nas propriedades consideradas obrigatórias para a entrega.
- **Sintetizabilidade:** o RTL deverá ser aceito pelo fluxo de síntese sem erros impeditivos ou inferências estruturais incompatíveis com a arquitetura especificada. Eventuais avisos relevantes deverão ser analisados e documentados.
- **Análise temporal:** a síntese deverá ser executada com a restrição inicial de clock de 100 MHz. Os relatórios deverão apresentar os resultados da análise temporal, incluindo eventuais violações. A execução da análise não implicará, por si só, garantia de fechamento temporal.
- **DFT e implementação física:** as etapas previstas de DFT, inserção de scan e implementação física deverão ser executadas de forma reproduzível na extensão suportada pelo ambiente. Limitações deverão ser explicitamente documentadas.
- **Rastreabilidade e documentação:** parâmetros de resolução, clock, PDK, biblioteca de células, versões das ferramentas, scripts, imagens e vetores de teste deverão ser identificados de modo a permitir a reprodução das análises.
- **Relatório de métricas:** os resultados de área, temporização e potência deverão ser reportados com base nos dados efetivamente produzidos pelas ferramentas. Não será considerada demonstrada qualquer meta de desempenho não sustentada pelos relatórios correspondentes.

6.9 Riscos e mitigação

Tabela 3: Riscos e estratégias de mitigação do Grupo 6c.

Risco	**Mitigação**
Ambiguidade no formato de pixels e sincronismo	Formalizar RGB888, largura da saída, sinais de validade, marcadores de linha/quadro e comportamento de reset antes da implementação definitiva.
Inconsistência entre Golden Model e RTL	Manter implementação independente do modelo Python, gerar vetores reproduzíveis e utilizar comparação automatizada por Scoreboard.
Erros de coordenadas no crop	Utilizar inicialmente imagens pequenas e padrões direcionados nos quais as coordenadas e bordas da região de interesse possam ser verificadas diretamente.
Complexidade excessiva no resize	Restringir a primeira versão ao método nearest-neighbor e priorizar downscaling, deixando algoritmos de interpolação mais complexos fora do escopo.
Erros de sincronismo após crop e resize	Definir explicitamente a regeneração dos marcadores de linha e quadro e utilizar assertions SVA e testes direcionados para validar o protocolo temporal.
Caminho aritmético crítico no grayscale	Avaliar os relatórios de síntese desde as primeiras versões e, se necessário, ajustar a decomposição das somas ou a estrutura de registros sem alterar a equação funcional especificada.
Uso excessivo de armazenamento interno	Evitar framebuffer completo e limitar a arquitetura a registradores e estruturas temporárias estritamente necessárias para o processamento em streaming.
Dificuldades no uso das ferramentas EDA	Validar antecipadamente o ambiente CIExpert, executar um fluxo mínimo ponta a ponta e registrar versões, bibliotecas e configurações utilizadas.
Indisponibilidade ou indefinição do PDK	Confirmar o PDK e a biblioteca de células educacionais antes da síntese definitiva e da implementação física.
Complexidade na inserção de scan	Executar incrementalmente as verificações DFT, analisar as regras de testabilidade e priorizar uma arquitetura baseada em registradores, documentando limitações identificadas.
Cobertura insuficiente de cenários de imagem	Incluir testes com padrões sintéticos, valores extremos de RGB, diferentes dimensões, regiões de crop, fatores de redução e thresholds, além de imagens reais para validação complementar.